Optimized Algorithms for ECPFL

Notation. $\mathcal{C}_i$ denotes the i-th client, $\mathcal{S}^{(t)}$ the active set in round t, $\bar{\mathcal{S}}^{(t)}$ the idle set, $\mathbf{w}_i$ the local model, $\mathbf{r}_i$ the residual vector, $\mathcal{K}^{(t)}$ the consensus pack set, and $\mathcal{Q}_j$ the server-side update queue for pack j.

Algorithm 1: ECPFL Main Procedure

Input: Local datasets $\{\mathcal{D}_i\}_{i=1}^{N}$, initial global model $\mathbf{w}^{(0)}$

Output: Final global model synchronized across clients

Parameters: Total rounds T, local epochs E, learning rate η, prefetch window R, sparsity k

; Initialization

KGC generates client keys $\{s_i\}_{i=1}^{N}$, global decryption key $S = \sum_i s_i$, and PRPG seed B;

Distribute (s_i, S) to each client $\mathcal{C}_i$, and distribute seed B to Server and all clients;

foreach *client* $\mathcal{C}_i \in \mathcal{C}$ **do**

  $\mathbf{w}_i \leftarrow \mathbf{w}^{(0)}, \quad \mathbf{r}_i \leftarrow \mathbf{0}, \quad t_{\text{last},i} \leftarrow 0;$

for $t = 1, 2, \ldots, T$ **do**

  ; Server: Client Selection

  Select active set $\mathcal{S}^{(t)}$ and idle set $\bar{\mathcal{S}}^{(t)}$;

  ; Active Clients: Local Training and Voting

  foreach *client* $\mathcal{C}_i \in \mathcal{S}^{(t)}$ **do** executed in parallel

    if $t - t_{last,i} > 1$ **then** prefetch model if previously idle

      Retrieve accumulated queues $\{\mathcal{Q}_j\}_{j=1}^{J}$ from Server;

      $\mathbf{w}_i \leftarrow \text{ECP-Sync}\big(\{\mathcal{Q}_j\}, S, \mathbf{w}_i, \eta\big)$; Alg. 3

    $\mathbf{w}_{i,\text{start}} \leftarrow \mathbf{w}_i$;

    $\mathbf{w}_i \leftarrow \mathsf{SGD}(\mathcal{D}_i, \mathbf{w}_i, \eta, E)$;

    $\mathbf{g}_i^{(t)} \leftarrow \frac{1}{\eta}\big(\mathbf{w}_{i,\text{start}} - \mathbf{w}_i\big)$;

    $(\mathbf{m}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)}) \leftarrow \text{ESP-Vote}\big(\mathbf{g}_i^{(t)}, \mathbf{r}_i, k\big)$; Alg. 2

    Submit masked vote $\mathbf{m}_i^{(t)}$ to the secure-aggregation primitive;

  ; Server: Secure Vote Aggregation and Consensus

  Collect masked votes $\{\mathbf{m}_i^{(t)}\}_{i\in\mathcal{S}^{(t)}}$;

  $\mathcal{K}^{(t)} \leftarrow \text{ESP-Consensus}\big(\{\mathbf{m}_i^{(t)}\}_{i\in\mathcal{S}^{(t)}}\big)$; server learns only aggregate votes

  Broadcast consensus set $\mathcal{K}^{(t)}$ to all active clients;

  ; Active Clients: Sparse Encryption and Upload

  foreach *client* $\mathcal{C}_i \in \mathcal{S}^{(t)}$ **do** executed in parallel

    $gap_i \leftarrow t - t_{\text{last},i}$;

    $(\{\mathsf{ct}_{i,j}^{(t)}\}, \mathbf{r}_i) \leftarrow \text{ESP-Enc}\big(\tilde{\mathbf{g}}_i^{(t)}, \mathcal{K}^{(t)}, \mathbf{r}_i, gap_i, s_i\big)$; Alg. 2

    Upload sparse ciphertexts $\{\mathsf{ct}_{i,j}^{(t)}\}$ to Server;

    Update local active round $t_{\text{last},i} \leftarrow t$;

  ; Server: Secure Aggregation and Queue Update

  Receive sparse ciphertexts $\{\mathsf{ct}_{i,j}^{(t)}\}_{i\in\mathcal{S}^{(t)}}$;

  $\{C_j^{(t)}\} \leftarrow \text{ESP-Agg}\big(\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \{\mathsf{ct}_{i,j}^{(t)}\}, \mathcal{K}^{(t)}\big)$; Alg. 2

  $\{\mathcal{Q}_j\} \leftarrow \text{ECP-QueueUpdate}\big(\{C_j^{(t)}\}, \mathcal{K}^{(t)}\big)$; Alg. 3

return globally synchronized model;

Algorithm 2: ESP Upstream Module

Parameters: Total packs J, sparsity k, scale Δ, decay factors γ_d, γ_s

Function *ESP-VOTE*$(\mathbf{g}_i^{(t)}, \mathbf{r}_i, k)$**:**

  $\tilde{\mathbf{g}}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{r}_i$;

  $\mathcal{T}_i \leftarrow$ Top-k indices of $\{\|\tilde{\mathbf{g}}_{i,j}^{(t)}\|_2^2\}_{j=1}^{J}$;

  Construct raw vote bitmap $\mathbf{v}_i^{(t)} \in \{0,1\}^J$ with $v_{i,j}^{(t)} = \mathbf{1}[j \in \mathcal{T}_i]$;

  $\mathbf{m}_i^{(t)} \leftarrow \textsc{SecAgg.Mask}\big(\mathbf{v}_i^{(t)}\big)$; mask vote before sending to server

  return $(\mathbf{m}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)})$;

Function *ESP-CONSENSUS*$(\{\mathbf{m}_i^{(t)}\}_{i\in\mathcal{S}^{(t)}})$**:**

  $\bar{\mathbf{v}}^{(t)} \leftarrow \textsc{SecAgg.Aggregate}\big(\{\mathbf{m}_i^{(t)}\}_{i\in\mathcal{S}^{(t)}}\big)$; only the sum of votes is revealed

  $\mathcal{K}^{(t)} \leftarrow$ Top-k indices of $\bar{\mathbf{v}}^{(t)}$;

  return $\mathcal{K}^{(t)}$;

Function *ESP-ENC*$(\tilde{\mathbf{g}}_i^{(t)}, \mathcal{K}^{(t)}, \mathbf{r}_i, gap_i, s_i)$**:**

  Derive global selection mask $\mathbf{U}^{(t)}$ where $U_j^{(t)} = \mathbf{1}[j \in \mathcal{K}^{(t)}]$;

  foreach $j \in \mathcal{K}^{(t)}$ **do**

    $\mathsf{pt}_{i,j}^{(t)} \leftarrow \mathsf{CKKS.Encode}\big(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta\big)$;

    $\mathsf{ct}_{i,j}^{(t)} \leftarrow \big[a^{(t)} \cdot s_i + \tilde{p} \cdot e_{i,j}^{(t)} + \mathsf{pt}_{i,j}^{(t)}\big]_q$; client-specific HE encryption

  $\mathbf{r}_i \leftarrow \gamma_d\, \gamma_s^{(gap_i - 1)} \Big[\tilde{\mathbf{g}}_i^{(t)} \odot \big(\mathbf{1} - \mathbf{U}^{(t)}\big)\Big]$; residual decay for unselected packs

  return $\big(\{\mathsf{ct}_{i,j}^{(t)}\}_{j\in\mathcal{K}^{(t)}}, \mathbf{r}_i\big)$;

Function *ESP-AGG*$(\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \{\mathsf{ct}_{i,j}^{(t)}\}, \mathcal{K}^{(t)})$**:**

  foreach $j \in \mathcal{K}^{(t)}$ **do**

    KGC locally generates null-compensatory ciphertexts $\{\mathsf{ct}_{k,j}^{(t)}\}$ for $k \in \bar{\mathcal{S}}^{(t)}$;

    privacy-preserving padding

    $C_j^{(t)} \leftarrow \sum_{i\in\mathcal{S}^{(t)}} \mathsf{ct}_{i,j}^{(t)} + \sum_{k\in\bar{\mathcal{S}}^{(t)}} \mathsf{ct}_{k,j}^{(t)}$; homomorphic aggregation

  return aggregated ciphertexts $\{C_j^{(t)}\}_{j\in\mathcal{K}^{(t)}}$;

Algorithm 3: ECP Download Module

Parameters: Prefetch window R, learning rate η

Function *ECP-QueueUpdate*($\{C_j^{(t)}\}, \mathcal{K}^{(t)}$)**:**

  foreach *pack* $j \in \{1, \ldots, J\}$ **do**

    if $j \in \mathcal{K}^{(t)}$ **then**

      Append $C_j^{(t)}$ to server queue $\mathcal{Q}_j$;

    else

      Append zero-ciphertext $[0]_q$ to queue $\mathcal{Q}_j$;

    if $|\mathcal{Q}_j| > R$ **then**

      Dequeue the stalest ciphertext from $\mathcal{Q}_j$; sliding window

  return updated queues $\{\mathcal{Q}_j\}_{j=1}^{J}$;

Function *ECP-Sync*($\{\mathcal{Q}_j\}, S, \mathbf{w}_i, \eta$)**:**

  foreach *pack* $j \in \{1, \ldots, J\}$ **do**

    $C_{\text{pre},j} \leftarrow \sum_{C \in \mathcal{Q}_j} C$; accumulate historical ciphertexts

    $\mathbf{m}_{\text{sum},j} \leftarrow \mathsf{CKKS.Dec}(S, C_{\text{pre},j})$;

  Reconstruct full global update vector $\mathbf{m}_{\text{sum}}$ from decoded packs $\{\mathbf{m}_{\text{sum},j}\}_{j=1}^{J}$;

  $\mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \cdot \mathbf{m}_{\text{sum}}$; synchronize local model

  return synchronized local model $\mathbf{w}_i$;